

EXPERIMENT : 10

Dynamic Memory Allocation

- 1) Write a program to apply bitwise OR, AND, and NOT operators on bit level

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node {
    int data ;
    struct Node * next;
};
```

```
int main ( ) {
    struct Node * head = NULL ;
    struct Node * second = NULL ;
    struct Node * third = NULL ;
```

```
    head = (struct Node *) malloc (sizeof (struct
                                     Node));
    second = (struct Node *) malloc (size of (struct
                                     Node));
    third = (struct Node *) malloc (size of (struct
                                     Node));
```

Experiment No. _____ Name: _____

PAGE NO.:

DATE: / /

head -> data = 10;
head -> next = second;

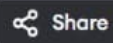
second -> data = 20;
second -> next = third;

third -> data = 30;
third -> next = NULL;

Output :- linked list : 10 -> 20 -> 30 -> NULL

Teacher's Signature: _____

main.c



Share

Run

Output

Clear

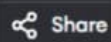
```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *next;
7 };
8
9 int main() {
10
11     struct Node *head = NULL;
12     struct Node *second = NULL;
13     struct Node *third = NULL;
14
15     head = (struct Node*)malloc(sizeof(struct Node));
16     second = (struct Node*)malloc(sizeof(struct Node));
17     third = (struct Node*)malloc(sizeof(struct Node));
18
19     head->data = 10;
20     head->next = second;
21
22     second->data = 20;
23     second->next = third;
24
25     third->data = 30;
26     third->next = NULL;
```

Linked List: 10 -> 20 -> 30 -> NULL

=== Code Execution Successful ===



main.c



Run

Output

Clear

```
16 second = (struct Node*)malloc(sizeof(struct Node));
17 third = (struct Node*)malloc(sizeof(struct Node));
18
19 head->data = 10;
20 head->next = second;
21
22 second->data = 20;
23 second->next = third;
24
25 third->data = 30;
26 third->next = NULL;
27
28 struct Node *temp = head;
29 printf("Linked List: ");
30 while (temp != NULL) {
31     printf("%d -> ", temp->data);
32     temp = temp->next;
33 }
34 printf("NULL\n");
35
36 free(head);
37 free(second);
38 free(third);
39
40 return 0;
41 }
```

```
^ Linked List: 10 -> 20 -> 30 -> NULL
```

```
=== Code Execution Successful ===
```


2) Write a program to insert item in middle of the linked list.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {
```

```
    int data;
```

```
    struct Node *next;
```

```
};
```

```
void insertMiddle (struct Node *head, int data,  
                  int position) {
```

```
    struct Node *newNode = (struct Node*)  
        malloc (sizeof (struct Node));
```

```
    newNode -> data = data;
```

```
    struct Node *temp = head;
```

```
    for (int i = 1; i < position; i++) {
```

```
        if (temp == NULL) {
```

```
            printf ("Position exceeds list length!\n");
```

```
            return;
```

```
        }
```

```
        temp = temp -> next;
```

```
    }
```

Experiment No. _____ Name: _____

PAGE NO.:

DATE: / /

```
newNode -> next = temp -> next;  
temp -> next = newNode;
```

```
}
```

```
void printList (struct Node *node) {  
    while (node != NULL) {  
        printf ("%d -> ", node->data);  
        node = node -> next;  
    }  
    printf ("NULL \n");  
}
```

```
int main () {
```

```
    struct Node *head = (struct Node*) malloc  
        (sizeof (struct Node));  
    struct Node *second = (struct Node*) malloc  
        (sizeof (struct Node));  
    struct Node *third = (struct Node*) malloc  
        (sizeof (struct Node));
```

```
    head -> data = 10;  
    head -> next = second;
```

```
    second -> data = 20;  
    second -> next = third;
```

Teacher's Signature: _____

Experiment No. _____ Name: _____

PAGE NO.:

DATE: / /

```
third -> data = 30;  
third -> next = NULL;
```

```
printf ("Original list : \n");  
printList (head);
```

```
InsertMiddle (head, 90, 2);
```

```
printf ("\n list After inserting in Middle:  
          \n");  
printList (head);
```

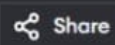
```
return 0;
```

```
}
```

Output :- Original list :
10 -> 20 -> 30 -> NULL

list After inserting in Middle:
10 -> 20 -> 90 -> 30 -> NULL

Teacher's Signature: _____



```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *next;
7 };
8
9 void insertMiddle(struct Node *head, int data, int position) {
10     struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
11     newNode->data = data;
12
13     struct Node *temp = head;
14
15     for (int i = 1; i < position; i++) {
16         if (temp == NULL) {
17             printf("Position exceeds list length!\n");
18             return;
19         }
20         temp = temp->next;
21     }
22
23     newNode->next = temp->next;
24     temp->next = newNode;
25 }
26
```

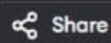
Original List:

10 -> 20 -> 30 -> NULL

List After Inserting in Middle:

10 -> 20 -> 99 -> 30 -> NULL

=== Code Execution Successful ===



```
27- void printList(struct Node *node) {
28-     while (node != NULL) {
29-         printf("%d -> ", node->data);
30-         node = node->next;
31-     }
32-     printf("NULL\n");
33- }
34-
35- int main() {
36-
37-     struct Node *head = (struct Node*)malloc(sizeof(struct Node));
38-     struct Node *second = (struct Node*)malloc(sizeof(struct Node));
39-     struct Node *third = (struct Node*)malloc(sizeof(struct Node));
40-
41-     head->data = 10;
42-     head->next = second;
43-
44-     second->data = 20;
45-     second->next = third;
46-
47-     third->data = 30;
48-     third->next = NULL;
49-
50-     printf("Original List:\n");
51-     printList(head);
52- }
```

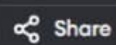
Original List:

10 -> 20 -> 30 -> NULL

List After Inserting in Middle:

10 -> 20 -> 99 -> 30 -> NULL

=== Code Execution Successful ===



```
34
35 int main() {
36
37     struct Node *head = (struct Node*)malloc(sizeof(struct Node));
38     struct Node *second = (struct Node*)malloc(sizeof(struct Node));
39     struct Node *third = (struct Node*)malloc(sizeof(struct Node));
40
41     head->data = 10;
42     head->next = second;
43
44     second->data = 20;
45     second->next = third;
46
47     third->data = 30;
48     third->next = NULL;
49
50     printf("Original List:\n");
51     printList(head);
52
53     insertMiddle(head, 99, 2);
54
55     printf("\nList After Inserting in Middle:\n");
56     printList(head);
57
58     return 0;
59 }
```

Original List:
10 -> 20 -> 30 -> NULL

List After Inserting in Middle:
10 -> 20 -> 99 -> 30 -> NULL

=== Code Execution Successful ===